

Project GEWN

ROADMAP.md

Development roadmap for the GEWN AI-Native Desktop Copilot Platform. This document outlines the phased build strategy from MVP to a full AI-native ecosystem.

Roadmap Philosophy

GEWN is built in deliberate phases — each one shippable, usable, and valuable on its own, while laying the foundation for everything that follows.

The guiding principle: **ship something real before adding something ambitious.**

Each phase has a clear entry condition (what must be true before it starts) and a clear exit condition (what must be true before the next phase begins). No phase begins until the previous one is stable.

Phase Overview

Phase 1 → MVP Core	Foundation: chat + memory + basic automation
Phase 2 → Voice AI	Natural voice interaction layer
Phase 3 → Desktop Automation	Deep OS-level control and workflow execution
Phase 4 → Vision & Context Awareness	Screen understanding and proactive assistance
Phase 5 → Autonomous Agent Workflows	Multi-step planning and self-executing tasks
Phase 6 → Plugin Ecosystem	Developer platform and third-party integrations
Phase 7 → Multi-Device AI Ecosystem	Cross-device sync, mobile, and enterprise

Phase 1 — MVP Core

Goal: Ship a working floating AI assistant that people actually want to use every day.

Timeline Estimate

10 – 16 weeks

Entry Condition

Architecture finalized, core stack selected, development environment set up.

Objectives

- Build a stable, usable floating desktop assistant.
 - Establish the core AI conversation loop with memory.
 - Deliver basic file management and app control.
 - Create the foundation that all future phases extend.
-

Features

Floating Desktop UI

- Always-on-top transparent overlay window
- Draggable, resizable assistant panel
- Minimize to a small icon in corner
- Dark futuristic UI with glassmorphism design
- Keyboard shortcut to show/hide (e.g. Ctrl+Space)

Conversational AI

- Natural language chat with streaming responses
- Markdown rendering with code syntax highlighting
- Conversation history within a session
- Follow-up question understanding
- Context injection from current session

Basic Memory

- Short-term session memory (Redis)
- Long-term preference memory (PostgreSQL)
- Simple semantic recall (ChromaDB, basic embeddings)

- User can tell GEWN to "remember" specific facts

Basic File Operations

- Create, rename, move, copy files and folders
- Open files in their default application
- Basic folder organization by file type
- Delete with confirmation prompt

Basic Application Control

- Launch applications by name
- Switch between open windows
- Close applications on request

Basic RAG (Document Chat)

- Upload PDF or text files
- Ask questions about uploaded documents
- Simple semantic search and retrieval

Authentication

- Local user account with encrypted storage
- Secure credential vault for API keys
- Session persistence

Technical Goals

- Electron + React frontend communicating with FastAPI backend via WebSocket
 - Streaming response pipeline from LLM to UI (token-by-token)
 - Initial LangChain integration for conversation management
 - ChromaDB set up with basic embedding pipeline
 - Docker-based development environment
 - GitHub Actions CI pipeline with lint and test gates
 - Logging and error tracking system established
-

Expected Challenges

Electron + WebSocket stability — Electron's IPC model combined with persistent WebSocket connections requires careful lifecycle management. Connections must survive sleep/wake cycles and window focus changes.

Streaming UX polish — Making streamed tokens feel smooth rather than janky requires debouncing and careful React state updates. This is harder than it looks.

Memory relevance tuning — Basic vector search will surface irrelevant memories if the embedding strategy is not well-designed early. Getting the chunking and retrieval logic right in Phase 1 saves significant rework later.

Cross-platform file paths — File operations that work on macOS may break on Windows due to path separator differences. Abstraction must be built in from day one.

Milestones

Milestone	Description
M1.1 — Overlay Shell	Electron window launches, floats on desktop, can be dragged and minimized
M1.2 — Chat Pipeline	User can send a message and receive a streamed AI response
M1.3 — Memory Online	Short-term and long-term memory writing and retrieval working end-to-end
M1.4 — File Ops	Create, move, rename, delete files via natural language commands
M1.5 — App Control	Open and close applications via chat
M1.6 — RAG Pipeline	Upload a PDF, ask questions, get contextual answers
M1.7 — MVP Stable	Full MVP used internally for one week without critical bugs

Phase 2 — Voice AI

Goal: Let users talk to GEWN naturally — hands-free, low-latency, and genuinely useful.

Timeline Estimate

8 – 12 weeks (begins after M1.7)

Entry Condition

MVP is stable. Chat, memory, and file operations are working reliably.

Objectives

- Add a complete voice interaction pipeline to GEWN.
 - Enable hands-free control of all Phase 1 features.
 - Build a real-time voice mode for extended conversations.
 - Ensure voice feels natural — not like talking to a phone IVR.
-

Features

Wake Word Detection

- "Hey GEWN" activates listening without touching the keyboard
- Visual pulse animation on activation
- Configurable custom wake word (future)
- Low CPU footprint — runs as a background microprocess

Speech-to-Text

- High-accuracy transcription via Whisper (local) or Deepgram (cloud)
- Supports multiple accents and natural speech patterns
- Handles filler words gracefully ("um", "uh" stripped before processing)
- Real-time transcript display as user speaks

Text-to-Speech

- Natural-sounding voice responses (ElevenLabs default; Coqui TTS local fallback)
- Adjustable speed and voice persona
- Emotion-aware tone variation (calm for info, alert for warnings)
- User can mute TTS and read responses instead

Real-Time Voice Mode

- Persistent open listening channel via WebSocket audio stream
- Interruption handling — user can speak while GEWN is responding
- Auto end-of-speech detection
- Graceful exit on command ("Stop", "That's all", shortcut key)

Voice-Specific UX

- Audio waveform visualization during listening
 - Transcript shown in real-time for verification
 - Voice command history panel
 - Fallback to text if STT confidence is below threshold
-

Technical Goals

- Audio streaming pipeline: mic → WebSocket → STT → LLM → TTS → speakers
 - Local Whisper model integration for offline/privacy use
 - Porcupine wake word integration running as a background subprocess
 - ElevenLabs API integration with audio buffer streaming
 - End-of-speech detection algorithm with configurable silence threshold
 - Voice pipeline must achieve < 1.5 second perceived latency (wake word to first audio output)
-

Expected Challenges

Latency — The pipeline has multiple hops: mic capture → STT → LLM → TTS → playback. Each adds latency. Achieving a natural-feeling response time requires careful parallelization — TTS should start generating from the first LLM tokens before the full response is ready.

Ambient noise handling — Wake word false positives in noisy environments (music, other voices, TV) are a known pain point with all voice assistants. Noise gate tuning and confidence thresholds need real-world testing.

Interruption model — Deciding when GEWN should yield mid-response is a genuinely hard UX problem. Too sensitive and it stops constantly; too stubborn and it ignores the user.

Local STT quality — Local Whisper models trade accuracy for privacy. The quality gap versus cloud STT is noticeable with accented speech or domain-specific terminology.

Milestones

Milestone	Description
M2.1 — Wake Word Active	"Hey GEWN" reliably activates the assistant with < 300ms detection latency
M2.2 — STT Pipeline	Spoken words transcribed accurately and displayed in real time
M2.3 — Voice Response	GEWN speaks responses aloud using TTS with natural pacing
M2.4 — Full Voice Loop	Complete voice interaction: wake → speak → respond → done
M2.5 — Real-Time Mode	Extended conversation without re-triggering wake word
M2.6 — Voice Stable	All Phase 1 features controllable by voice; used internally for one week

Phase 3 — Desktop Automation

Goal: Give GEWN real hands — let it control the computer the way a skilled human would.

Timeline Estimate

10 – 14 weeks (begins after M2.6)

Entry Condition

Voice pipeline is stable. GEWN can handle compound natural language requests reliably.

Objectives

- Enable GEWN to perform complex, multi-step desktop tasks autonomously.

- Build a browser automation capability for web-based workflows.
 - Create a reusable workflow preset system.
 - Establish the permission and safety framework for all future automation.
-

Features

Advanced File Management

- Semantic file organization (group files by content, not just type)
- Bulk rename with pattern-based rules
- Duplicate file detection and cleanup
- Archive and compress folders on request
- Smart folder watching with automatic triggers

Application Automation

- Automate sequences within applications (not just open/close)
- Window layout management (arrange windows in defined layouts)
- Clipboard management and history
- System shortcut automation
- Startup workflow configuration

Browser Automation

- Search and navigate websites on command
- Extract and summarize web page content
- Fill and submit forms (with permission)
- Download files from the web
- Multi-tab research workflows

Workflow Presets

- Save any executed workflow as a named preset
- Recall presets by name: "Run my morning routine"
- Edit preset steps through natural language
- Share presets as exportable JSON files

- Scheduled workflow execution (time-based or event-based triggers)

Permission & Safety Framework

- 4-tier risk classification for all automation actions
 - Full approval dialog for high-risk operations
 - Undo buffer for reversible actions
 - Automation audit log (what ran, when, what changed)
 - Sandbox mode for testing workflows before executing
-

Technical Goals

- PyAutoGUI integration for OS-level mouse/keyboard automation
 - Playwright integration for reliable cross-browser web automation
 - Workflow execution engine with step-by-step state management
 - Robust error handling and partial-rollback system
 - Cross-platform compatibility testing (macOS, Windows)
 - Automation test suite covering core workflow scenarios
-

Expected Challenges

OS permission walls — macOS Accessibility permissions and Windows UAC create real friction. Users must manually grant access, and this flow needs to be smooth and clearly explained.

Application fragility — Automating third-party applications via UI means relying on element positions and labels that can change with any app update. Playwright's web automation is more stable than GUI automation, but desktop app automation is brittle.

Workflow complexity — A workflow that works the first time may fail when the context changes (different window position, file already exists, app already open). Defensive step execution with smart condition checking is hard to get right.

Safety reputation — One accidental deletion of an important file erodes user trust permanently. The safety and permission framework must be bulletproof before any broad release.

Milestones

Milestone	Description
M3.1 — Browser Agent	GEWN can navigate to a URL, read content, and summarize a web page
M3.2 — App Automation	GEWN can execute multi-step sequences within applications
M3.3 — Workflow Presets	Users can save, name, and replay any workflow
M3.4 — Scheduler	Workflows execute on time-based triggers reliably
M3.5 — Safety Framework	All automation passes through risk classifier; no destructive action runs without approval
M3.6 — Automation Stable	End-to-end workflow automation used daily by internal team without critical incidents

Phase 4 — Vision & Context Awareness

Goal: Give GEWN eyes — let it understand what's on screen and help without being asked.

Timeline Estimate

10 – 14 weeks (begins after M3.6)

Entry Condition

Automation framework is stable and safe. Permission system is proven reliable.

Objectives

- Enable GEWN to understand and analyze screen content in real time.
- Build proactive assistance triggered by what GEWN observes, not just what the user asks.
- Create visual overlay guidance for complex UI navigation.
- Integrate coding error detection for developer users.

Features

Screen Capture & OCR

- Low-frequency passive screen monitoring (opt-in, privacy-respecting)
- High-accuracy OCR via PaddleOCR with Tesseract fallback
- Active-window focus detection for targeted capture
- User-triggered "analyze this screen" command
- Screen region selection for precise queries

Multimodal Screen Understanding

- Send screenshot + OCR text to multimodal LLM for combined analysis
- Understand UI elements: buttons, forms, menus, panels
- Interpret error messages, logs, and terminal output
- Analyze document layouts and spreadsheet structures
- Detect context switches (user moved from coding to browsing)

Coding Error Detection

- Passive monitoring for error patterns in code editors (VS Code, terminal)
- Detect stack traces, linter warnings, compile errors, test failures
- Surface explanation and fix suggestion without user asking
- One-click "Apply Fix" action

Visual Overlay Guidance

- Render transparent overlay on top of any screen region
- Annotate UI elements with labels and arrows
- Step-by-step guided overlays for complex workflows
- Highlight specific elements the user should interact with
- Overlays dismiss automatically or on user command

Proactive Context Assistance

- GEWN notices what the user is doing and offers relevant help
- "I see you're filling out a form — want me to autofill from your saved profile?"
- "You've been on this page for a while — want a summary?"

- Proactive suggestions are non-intrusive (small notification, not a pop-up)
 - User can disable proactive mode entirely
-

Technical Goals

- Efficient screenshot pipeline that doesn't impact system performance (< 2% CPU overhead for passive monitoring)
 - PaddleOCR integration with GPU acceleration where available
 - Multimodal API integration (GPT-4V / Gemini Vision)
 - Overlay rendering system using Electron's transparent window layering
 - Context switch detection using active window event listeners
 - Privacy guarantee: no screen data stored or transmitted without explicit user action
-

Expected Challenges

Performance — Continuous screen capture and OCR processing is CPU and memory intensive. The pipeline must be heavily optimized: only capture on significant screen changes, process only the active region, and defer heavy LLM calls to when the user is idle.

Accuracy of proactive suggestions — Offering help at the wrong moment is annoying. The signal-to-noise ratio of proactive suggestions needs careful calibration. Too many suggestions and users disable the feature; too few and it feels passive.

Privacy perception — Screen monitoring, even opt-in, makes users nervous. GEWN needs very clear visual indicators (a small always-visible icon) showing when screen monitoring is active, and it must be trivial to pause or disable.

OCR limitations — OCR on high-DPI screens, stylized fonts, or non-English text can produce unreliable output. Graceful degradation (visual-only LLM fallback) must be in place.

Milestones

Milestone	Description
M4.1 — OCR Pipeline	GEWN accurately extracts text from any on-screen content on demand

Milestone	Description
M4.2 — Screen Understanding	GEWN can answer questions about what is currently visible on screen
M4.3 — Error Detection	Coding errors surface automatically with explanation and fix suggestion
M4.4 — Overlay System	Visual overlays render correctly and dismiss cleanly
M4.5 — Proactive Mode	GEWN offers contextually relevant suggestions based on observed activity
M4.6 — Vision Stable	Vision features used daily; privacy controls validated; performance benchmarks met

Phase 5 — Autonomous Agent Workflows

Goal: Let GEWN plan and execute complex, multi-step tasks without hand-holding.

Timeline Estimate

12 – 16 weeks (begins after M4.6)

Entry Condition

Vision and context awareness are stable. GEWN reliably understands user intent from complex requests.

Objectives

- Build a multi-agent orchestration system with specialized agents.
 - Enable GEWN to plan and execute multi-step tasks autonomously.
 - Develop a planning engine that can decompose complex goals into executable steps.
 - Build agent memory so workflows improve with repetition.
-

Features

Specialized AI Agents

Agent	Responsibilities
Coding Agent	Code generation, debugging, refactoring, test writing
Research Agent	Web search, PDF summarization, insight synthesis
Automation Agent	Desktop workflow execution, file management, app control
Memory Agent	Embedding generation, context retrieval, knowledge organization
Scheduling Agent	Calendar integration, reminders, time-based workflow triggers

Planning Engine

- Decompose high-level goals into ordered, executable steps
- Select the right agent or tool for each step
- Handle step dependencies and sequencing
- Adapt the plan if a step fails (replan, not just stop)
- Show plan to user before executing; allow editing

Multi-Agent Coordination

- Agents communicate through a shared context object
- Agent handoffs are clean and logged
- Parallel agent execution where steps are independent
- Master orchestrator agent manages routing and conflict resolution

Autonomous Task Execution

- Run extended workflows in the background while user works
- Progress shown in a non-intrusive status bar
- User can pause, modify, or cancel any running workflow
- Completed workflows summarized with a brief report

Agent Memory & Learning

- Agents remember which approaches worked and which failed
 - Successful workflows stored as reusable templates
 - Agents improve their tool selection over repeated use
 - Feedback loop: user rates outcome → stored as training signal
-

Technical Goals

- LangGraph implementation for stateful multi-agent orchestration
 - Agent routing system with intent-to-agent classification
 - Shared agent context store (Redis for hot state, PostgreSQL for persistence)
 - Parallel agent execution using async task workers
 - Comprehensive agent execution logging for debugging and audit
 - Failure recovery logic with configurable retry and fallback strategies
-

Expected Challenges

Agent coordination complexity — Multi-agent systems fail in subtle ways: agents disagree, produce conflicting outputs, or get stuck in loops. The orchestration layer must have strict timeouts, circuit breakers, and escalation paths.

Planning accuracy — LLM-generated plans look plausible but can include impossible steps, wrong tool selections, or logical contradictions. Plans must be validated before execution, not just displayed.

Long-running task stability — A workflow that takes 10 minutes involves dozens of individual actions across potentially unstable system states (app crashed, network dropped, file moved). Checkpoint and resume logic is essential.

User trust in autonomy — Users need to trust that GEWN won't do something unexpected during autonomous execution. Clear pre-execution plans, live step visibility, and easy cancellation are non-negotiable trust anchors.

Milestones

Milestone	Description
M5.1 — Agent Routing	Intent is correctly routed to the appropriate specialized agent
M5.2 — Planning Engine	GEWN produces correct multi-step plans for complex compound requests
M5.3 — Multi-Agent Flow	Two or more agents coordinate on a single task end-to-end

Milestone	Description
M5.4 — Background Execution	Long workflows run in background with live status and cancellation
M5.5 — Failure Recovery	Agents handle mid-workflow failures gracefully without losing progress
M5.6 — Agent System Stable	Autonomous workflows execute reliably across a diverse set of real-world tasks

Phase 6 — Plugin Ecosystem

Goal: Open GEWN to the world — let developers build on top of it.

Timeline Estimate

12 – 16 weeks (begins after M5.6)

Entry Condition

Autonomous agent framework is stable. Core API is clean and well-documented.

Objectives

- Build a developer-facing plugin SDK for extending GEWN.
 - Create a third-party integration library for popular tools.
 - Launch an early developer program and plugin registry.
 - Establish the foundation for a plugin marketplace.
-

Features

Plugin SDK

- Python SDK for building GEWN plugins
- Plugin manifest format (`manifest.json`) with declared capabilities
- Plugin can add: commands, agents, workflows, UI panels, integrations
- Hot-reload during development

- Sandboxed execution with declared permission scopes

Plugin Structure

my-plugin/

— manifest.json	# Name, version, permissions, entry point
— commands/	# Custom commands the plugin adds
— agents/	# Custom AI agents
— workflows/	# Predefined workflow templates
— integrations/	# External API connections

Core Integrations (built by GEWN team)

- GitHub — PR summaries, issue tracking, code review assistance
- Notion — page creation, search, database updates
- Gmail — email summarization, draft generation, send on command
- Slack / Discord — send messages, summarize channels, set reminders
- Google Calendar — create events, check schedule, reschedule meetings
- Linear / Jira — create tickets, update status, sprint summaries

Developer Tools

- Plugin CLI for scaffolding new plugins
- Local plugin testing environment
- Plugin event hooks (on message, on screen change, on schedule)
- Plugin API documentation and cookbook
- Community plugin repository (GitHub-based, pre-marketplace)

Plugin Registry (v1)

- Installable plugins via GEWN settings panel
- Plugin ratings and descriptions
- One-click install and uninstall
- Automatic update notifications

Technical Goals

- Plugin loader with sandboxed subprocess execution

- Permission model: plugins declare required scopes; user approves on install
 - Plugin event bus with pub/sub architecture
 - REST and WebSocket APIs exposed to plugins
 - Plugin developer documentation site
 - Automated plugin validation pipeline (security scan + API compatibility check)
-

Expected Challenges

Security — Plugins run third-party code inside the user's desktop environment. Sandboxing must be robust without being so restrictive that plugins can't do useful things. Getting this balance right is hard and critical.

Developer experience — A plugin ecosystem only grows if developers find it easy and rewarding. The SDK, docs, and tooling must be genuinely excellent — not just technically correct.

Plugin quality control — Without a strong review process, the registry will accumulate broken or low-quality plugins that damage GEWN's reputation. Early curation is necessary even before automated review scales.

API stability — Once external plugins depend on GEWN's APIs, breaking changes become expensive. API versioning and a clear deprecation policy must be in place before the SDK is public.

Milestones

Milestone	Description
M6.1 — Plugin Loader	GEWN can install and execute a local plugin from manifest
M6.2 — SDK Alpha	Python SDK published with core command and workflow extension points
M6.3 — Core Integrations	GitHub, Notion, and Gmail integrations working via plugin system
M6.4 — Developer Docs	Full SDK documentation, quickstart guide, and 3+ example plugins published
M6.5 — Plugin Registry	Users can browse and install community plugins from within GEWN

Milestone	Description
M6.6 — Ecosystem Launch	Early developer program launched; 10+ community plugins available

Phase 7 — Multi-Device AI Ecosystem

Goal: GEWN follows the user everywhere — desktop, mobile, and cloud, all in sync.

Timeline Estimate

14 – 20 weeks (begins after M6.6)

Entry Condition

Plugin ecosystem is stable. Core platform has consistent, reliable performance. Infrastructure is ready for multi-tenant workloads.

Objectives

- Build cross-device memory and workflow synchronization.
 - Launch a GEWN mobile companion app.
 - Enable local offline AI execution for power users.
 - Build the foundation for enterprise team features.
-

Features

Cloud Memory Sync

- User's memory, preferences, and workflows sync across devices
- Conflict resolution when the same memory is updated on two devices
- Selective sync — user controls which memory categories sync
- End-to-end encrypted sync (keys never leave the user's devices)

Mobile Companion App

- iOS and Android app with core GEWN features

- Voice interaction on mobile (same pipeline, mobile-optimized)
- Receive and respond to GEWN notifications from desktop
- Trigger desktop workflows from mobile ("Start my work setup")
- Mobile-specific features: camera input, location context

Local AI Infrastructure

- Ollama integration for running open-source LLMs on-device
- Privacy Mode: all processing stays on-device, zero cloud calls
- Model management UI: download, switch, and update local models
- Performance profiling: automatically selects best model for device hardware
- Supports quantized models for machines without dedicated GPU

Cross-Device Workflow Continuity

- Start a research task on desktop, continue on mobile
- Notifications routed intelligently by device and time of day
- Shared clipboard between desktop and mobile via GEWN
- Handoff protocol: state of current task transferred between devices

Enterprise Foundation (v1)

- Team workspace with shared agents and workflows
- Admin controls: permission management, audit logs, policy enforcement
- SSO integration (Google Workspace, Microsoft 365)
- Usage analytics dashboard for team administrators
- Data residency options for compliance requirements

Technical Goals

- End-to-end encrypted sync protocol with conflict resolution
- React Native mobile app sharing core logic with desktop where possible
- Ollama integration with model download management and automatic model selection
- Push notification infrastructure (APNs, FCM) for mobile
- Multi-tenant architecture for team workspaces
- SOC 2 Type II compliance work begins this phase

Expected Challenges

Sync complexity — Distributed state is one of the hardest problems in software engineering. Memory objects updated on two devices simultaneously need deterministic conflict resolution without data loss.

Mobile performance — Running GEWN's AI pipeline on a mobile device — even a companion version — requires aggressive optimization. Many features that feel instant on desktop will feel slow on mobile without careful offloading.

Local LLM quality gap — Even the best local models in 2024–2025 lag behind frontier cloud models significantly on complex reasoning tasks. GEWN must set clear user expectations and gracefully route complex tasks to cloud when local models are insufficient.

Enterprise requirements — Enterprise buyers have security, compliance, and procurement requirements that are entirely different from individual users. Building enterprise features too early wastes engineering time; building them too late means losing enterprise deals. Phase 7 begins this work without letting it dominate the roadmap.

Scope creep — Phase 7 is the broadest phase on the roadmap. The risk of trying to do everything at once is high. Strict prioritization — mobile before enterprise, sync before local AI — is essential.

Milestones

Milestone	Description
M7.1 — Cloud Sync	Memory and preferences sync reliably across two desktop instances
M7.2 — Mobile Alpha	iOS app with voice chat and workflow trigger working end-to-end
M7.3 — Local AI Mode	Ollama integration working; Privacy Mode fully operational on desktop
M7.4 — Cross-Device Continuity	Task started on desktop can be resumed on mobile and vice versa
M7.5 — Enterprise Alpha	Team workspace with shared workflows and admin controls available to early partners

Milestone	Description
M7.6 — Ecosystem v1	All 7 phases stable, documented, and in active use by real users

Ongoing Work (All Phases)

The following work runs in parallel with every phase and is never "done":

- Performance** — Response latency, memory retrieval speed, and UI responsiveness are continuously benchmarked and improved.
- Security** — Penetration testing, dependency audits, and permission model reviews run on a regular cadence.
- Reliability** — Crash reporting, automated error recovery, and graceful degradation are improved continuously as real-world usage surfaces new failure modes.
- Accessibility** — Keyboard navigation, screen reader compatibility, and contrast/font settings are maintained throughout.
- Documentation** — User guides, developer docs, and architecture references are kept current with every feature addition.

Risk Register

Risk	Likelihood	Impact	Mitigation
AI API cost overruns at scale	High	High	Local model fallbacks, caching, efficient prompt design
Automation causes data loss	Low	Critical	Multi-tier permission system, undo buffers, audit logs
Screen monitoring triggers privacy backlash	Medium	High	Opt-in only, clear indicators, local-first processing
LLM quality regression from provider	Medium	High	Multi-provider support (OpenAI + Gemini + local)
Plugin ecosystem introduces security vulnerabilities	Medium	High	Sandboxed execution, permission scopes, code review

Risk	Likelihood	Impact	Mitigation
Cross-platform inconsistency (Mac vs Windows)	High	Medium	Platform abstraction layer built in Phase 1
Team scope creep delays core features	High	Medium	Strict phase gates; each phase must exit before next begins

Success Definition

GEWN is successful when a user who has used it for 30 days says:

"I don't know how I worked without this."

That means GEWN must become genuinely indispensable — not impressive in a demo, but reliable and useful in the daily friction of real work. Every phase is evaluated against that standard.

Document version: 1.0 — updated as phases progress and priorities evolve.